07 기본 커널 함수

- User memory access

unsigned long copy_to_user (void __user *to, const void *from, unsigned long n)

유저 메모리에 데이터를 복사

unsigned long copy_from_user (void *to, const __user *from, unsigned long n)

유저 메모리에서 데이터를 복사

사용하는 이유 : 사용자버퍼가 읽거나 쓰지 못하는 상태일 수도 (스왑)
유저가 악의적으로 커널 메모리를 쓰거나 읽거나 쓰려고 시도하는 것을 방지

- memory allocation

void *vmalloc (크기) → void vfree (주소)
가상공간 주소에서 연속적인 메모리 할당

void *kmalloc (크기, flag) → void kfree (objp)
실제 물리적으로 연속된 메모리 할당

flag: GFP_KERNEL : 메모리 부족시 sleep
GFP_ATOMIC : 슬립X, 메모리 부족시 실패
GFP_DMA : DMA 장치 전용

- linked list

커널 내장 연결 리스트 형태

값은 없음 (동시 접근시 개발자가 락처리)

사용자의 구조체 안에 list_head 구조체가 포함된 형태

```
struct list_head { struct list_head *next, prev }
```

초기화는 void INIT_LIST_HEAD (포인터)

list_add : head ⇒ 추가 → 노드 → 노드 (스택)

list_add_tail : 추가 → 노드 → 노드 → head (큐)

list_del : 연결을 끊음

list_del_init : list_head 구조체 초기화 까지 안전하게 재사용

list_for_each : 매크로 함수

(pos, head) pos는 현재 보고 있는 위치

```
{
    안에 list_entry (pos, 노드 구조체, list_head 구조체)
        ↙
      이것으로 우리 노드 구조체 가져옴.
}
```

list_for_each_safe : 내가 노드 삭제 해버려서 순회해도 안전

(pos, n, head) 다음 엔트리를 n에 저장

- Spin lock

정적선언 static DEFINE_SPINLOCK (my_lock);
구조체 메모에서 동적선언 가능

spin_lock (락)
spin_unlock (락)

잠들지 않고 락을 얻을 때까지 busy waiting

- Delay

void udelay 마이크로초 busy waiting
void mdelay 밀리초 busy waiting

void msleep 밀리초 수면, 인터럽트 컨텍스트에서 사용금지
schedule_timeout jiffies 커널타이머 틱 동안 수면
wait_event_timeout 조건이 참이 되거나, 타임아웃이 걸과 될때 까지 수면